Electria

Entity Relationship Diagram

Peer-to-peer personal electric vehicle (PEV) rental marketplace | Logical data model | 12 entities: 10 MVP + 2 stretch
Group 15 | Software Engineering-CTW | Fall 2026 | Target stack: PostgreSQL + Prisma (Next.js)

VEHICLE_CATEGORY
e-bike, e-scooter, e-skateboard...

PK	**category_id**	UUID
UK	name	VARCHAR(40)
	description	TEXT ?

VEHICLE
The physical PEV and its owner

PK	**vehicle_id**	UUID
FK	owner_id	UUID
FK	category_id	UUID
	brand	VARCHAR(60)
	model	VARCHAR(80)
	model_year	SMALLINT ?
	description	TEXT
	allowed_use	VARCHAR(20)
	condition_status	VARCHAR(20)

LISTING
Public offer + pickup location

PK	**listing_id**	UUID
FK UK	vehicle_id	UUID
	title	VARCHAR(120)
	daily_rate	DECIMAL(10,2)
	currency	CHAR(3)
	pickup_city	VARCHAR(80)
	pickup_address	VARCHAR(255)
	pickup_lat	DECIMAL(9,6) ?
	pickup_lng	DECIMAL(9,6) ?
	listing_status	VARCHAR(20)
	created_at	TIMESTAMPTZ

LISTING_PHOTO
Photo gallery per listing

PK	**photo_id**	UUID
FK	listing_id	UUID
	photo_url	VARCHAR(500)
	sort_order	SMALLINT
	is_cover	BOOLEAN

AVAILABILITY_BLOCK
Renter-declared unavailable time

PK	**block_id**	UUID
FK	listing_id	UUID
	start_at	TIMESTAMPTZ
	end_at	TIMESTAMPTZ
	reason	VARCHAR(120) ?

MAINTENANCE_LOG
STRETCH
Maintenance & incident history

PK	**log_id**	UUID
FK	vehicle_id	UUID
	entry_type	VARCHAR(20)
	notes	TEXT
	logged_at	TIMESTAMPTZ

USER
One account, Rider and/or Renter

PK	**user_id**	UUID
	full_name	VARCHAR(100)
UK	email	VARCHAR(254)
	password_hash	VARCHAR(255)
	phone	VARCHAR(30) ?
	home_city	VARCHAR(80)
	date_of_birth	DATE
	role	VARCHAR(10)
	id_verified	BOOLEAN
	created_at	TIMESTAMPTZ
	account_status	VARCHAR(20)

BOOKING
One rider, one listing, one time window

PK	**booking_id**	UUID
FK	listing_id	UUID
FK	rider_id	UUID
	start_at	TIMESTAMPTZ
	end_at	TIMESTAMPTZ
	status	VARCHAR(20)
	daily_rate_at_booking	DECIMAL(10,2)
	service_fee	DECIMAL(10,2)
	total_amount	DECIMAL(10,2)
	currency	CHAR(3)
	created_at	TIMESTAMPTZ

PAYMENT
Stripe test-mode payment attempts

PK	**payment_id**	UUID
FK	booking_id	UUID
UK	stripe_payment_id	VARCHAR(120)
	amount	DECIMAL(10,2)
	currency	CHAR(3)
	status	VARCHAR(20)
	paid_at	TIMESTAMPTZ ?

MESSAGE
Chat tied to a booking

PK	**message_id**	UUID
FK	booking_id	UUID
FK	sender_id	UUID
	body	TEXT
	sent_at	TIMESTAMPTZ

REVIEW
Two-way rating after a rental

PK	**review_id**	UUID
FK	booking_id	UUID
FK	reviewer_id	UUID
FK	reviewee_id	UUID
	rating	SMALLINT
	comment	TEXT ?
	created_at	TIMESTAMPTZ

DISPUTE
STRETCH
Flagged issue, resolved by an admin

PK	**dispute_id**	UUID
FK	booking_id	UUID
FK	raised_by	UUID
FK	resolved_by	UUID ?
	reason	TEXT
	status	VARCHAR(20)
	created_at	TIMESTAMPTZ
	resolved_at	TIMESTAMPTZ ?

classifies · *offered as* · *has photos* · *blocked time* · *owns* · *has log* · *receives* · *paid via* · *books as Rider* · *booking thread* · *up to 2 reviews* · *may be disputed* · *sends* · *reviewer / reviewee* · *raises / resolves (admin)*

CARDINALITY

- exactly one
- zero or one
- zero or many

Read each line from the '1' side to the 'many' side, e.g. one USER owns many VEHICLEs; each VEHICLE has one owner.

KEYS

- PK Primary key
- FK Foreign key
- UK Unique key
- ? nullable

STRETCH

- Dashed outline / italic field = stretch requirement

CONVENTIONS

Rider = books a PEV; Renter = owns and lists one. One USER can be both.
role: rider / renter / both / admin. Statuses are controlled value lists.
BOOKING.status: requested, confirmed, completed, cancelled.

Relationships & design decisions

Electria | Group 15 | How the data model supports the use case diagram and functional requirements (FR1-FR9, S1-S5)

RELATIONSHIP REFERENCE

USER to VEHICLE	`1 : 0..*`	A user (in the Renter role) owns many PEVs; each PEV has exactly one owner.
VEHICLE_CATEGORY to VEHICLE	`1 : 0..*`	A category classifies many vehicles; each vehicle has one category.
VEHICLE to LISTING	`1 : 0..1`	A vehicle has at most one listing. The listing is archived, not deleted, so history stays intact.
LISTING to LISTING_PHOTO	`1 : 0..*`	A listing has a gallery of photos; one is flagged as the cover.
LISTING to AVAILABILITY_BLOCK	`1 : 0..*`	The Renter blocks out dates when the PEV cannot be rented.
LISTING to BOOKING	`1 : 0..*`	A listing receives many booking requests over time.
USER to BOOKING (rider_id)	`1 : 0..*`	A user makes many bookings as a Rider. A user cannot book their own vehicle.
BOOKING to PAYMENT	`1 : 0..*`	One booking can have several payment attempts; at most one may succeed.
BOOKING to MESSAGE	`1 : 0..*`	Messages always belong to a booking, giving each rental its own thread.
USER to MESSAGE (sender_id)	`1 : 0..*`	Each message has exactly one sender; the receiver is the other party on the booking.
BOOKING to REVIEW	`1 : 0..2`	Rider and Renter each leave one review after a completed rental.
USER to REVIEW (reviewer / reviewee)	`1 : 0..*`	Two foreign keys to USER: who wrote the review and who it is about.
VEHICLE to MAINTENANCE_LOG [stretch]	`1 : 0..*`	The Renter keeps a maintenance and incident history per PEV.
BOOKING to DISPUTE [stretch]	`1 : 0..*`	A booking may be flagged; an admin reviews and resolves the dispute.
USER to DISPUTE (raised_by / resolved_by) [stretch]	`1 : 0..*`	Links the person who raised it and the admin who resolved it.

DESIGN DECISIONS & TRACEABILITY

1 One account, two roles — FR1

USER has a role (rider / renter / both / admin). A user becomes a Renter by listing a vehicle and a Rider by making a booking, so no separate role tables are needed.

2 Vehicle vs. listing — FR2, FR3

VEHICLE is the physical PEV; LISTING holds the price, pickup city and public offer. Pausing or archiving a listing never loses its booking history.

3 Photos and availability — FR2

Listings take many photos (LISTING_PHOTO) and Renter-declared unavailable periods (AVAILABILITY_BLOCK) that feed the search and booking checks.

4 Search and filter — FR3, NFR

Index listing_status, pickup_city and category_id so search by location, price and vehicle type stays within the 2-second target.

5 Booking lifecycle and price snapshot — FR4, FR7

BOOKING.status moves requested, confirmed, completed or cancelled. The agreed daily rate, fee and total are copied onto the booking so later price edits do not change it.

6 Safe payments — FR5, NFR

Stripe handles card data, so PAYMENT stores only the provider reference and status. A booking becomes confirmed only after a succeeded payment, written in one transaction.

7 No double booking — FR4, NFR

Reject a booking whose time range overlaps another confirmed booking or an availability block on the same listing. Check inside a transaction.

8 Messages tied to bookings — FR6, S5

MESSAGE references a booking and a sender. Stretch: live updates can be added without changing the schema.

9 Two-way reviews — FR8, FR9

REVIEW has reviewer_id and reviewee_id, unique per (booking_id, reviewer_id). Only allowed once the booking is completed. Booking history is simply the user's BOOKING rows.

10 Stretch features stay optional — S1-S4

id_verified (S1), MAINTENANCE_LOG (S2) and DISPUTE (S4) can be dropped without touching the MVP tables. Earnings (S3) is a query over completed bookings and succeeded payments, so it needs no table.

FR = functional requirement, S = stretch requirement, NFR = non-functional requirement, from the Electria use case document. Out of scope: payouts, deposits, taxes, insurance, refunds.